МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
ФАКУЛЬТЕТ БИЗНЕСА
Кафедра экономической информатики

ОТЧЁТ О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ № 4

Тема: «Асинхронная модель исполнения компонента»

Выполнил: студент группы ФБИ-__
ФИО: _____

Проверил: _____

Новосибирск, 2026

1. Раздел I. Описание предметной области

Разрабатывается агрегатор поиска авиаперелётов. Клиент задаёт аэропорт вылета, аэропорт прилёта и дату. Сервис получает рейсы из собственной базы PostgreSQL и от трёх поставщиков. На запрос к каждому поставщику отводится не более 2 секунд вместе с одним повтором после ошибки через 200 мс. Рейсы неответившего поставщика исключаются, его имя включается в ответ. Если ни один поставщик не ответил и в собственной базе нет рейсов, сервис возвращает сообщение о временной недоступности поиска.

Сервис строит прямые маршруты и маршруты не более чем с двумя пересадками. Все рейсы вылетают в дату поиска; пересадка выполняется в аэропорту прилёта предыдущего рейса; ожидание между рейсами составляет от 1 до 6 часов; аэропорты не повторяются. Цена маршрута равна сумме цен рейсов. В ответ включаются не более 20 самых дешёвых маршрутов.

2. Раздел II. Анализ нагрузки

2.1. Операции обработки запроса

Операция	На что тратится время	Длительность	С чем может выполняться одновременно
Проверка параметров запроса	Вычисления	менее 1 мс	Перед чтением БД и опросом поставщиков
Чтение рейсов собственной БД	Ожидание БД	около 5 мс (оценка)	До опроса поставщиков в синхронной версии; одновременно с ними в версии 4.2
Запрос поставщику 1	Ожидание сети	среднее около 325 мс с учётом повторов; предел 2 с	Последовательно после БД в версии 3.3/4.1; параллельно с остальными в 4.2
Запрос поставщику 2	Ожидание сети	среднее около 535 мс с учётом повторов; предел 2 с	То же
Запрос поставщику 3	Ожидание сети	среднее около 965 мс с учётом повторов; предел 2 с	То же; 10% ответов задерживаются на 3 с и попадают в тайм-аут
Построение маршрутов и сортировка	Вычисления	около 61 мс на тестовом наборе 3000 рейсов	После получения всех доступных рейсов

Формирование JSON-ответа	Вычисления	около 1 мс (оценка)	После построения маршрутов
Пауза перед повтором	Ожидание таймера	200 мс при ошибке, если срок позволяет	Внутри запроса к одному поставщику; другие запросы могут выполняться параллельно в версии 4.2

Значения времени поставщиков получены моделированием 400 000 запросов на основе параметров таблицы 1: равномерная задержка в диапазоне ±30%, вероятность ошибки 500 равна 5%, один повтор через 200 мс, общий срок 2 секунды. Для поставщика 3 дополнительно моделировались 10% ответов с задержкой 3 секунды. Это расчётная оценка, не результат нагрузочного замера реального сервиса. Время чтения БД 5 мс и формирования JSON 1 мс взяты как оценочные значения из дополнительных материалов. Построение маршрутов измерено локально на сгенерированном наборе из 3000 рейсов.

2.2. Диаграммы последовательности UML и среднее время

Диаграмма последовательного опроса сохранена в diagrams/search_sequential.puml. Сервис сначала читает собственную БД, затем опрашивает поставщиков 1, 2 и 3 по очереди. При ошибке выполняется пауза 200 мс и один повтор, если общий двухсекундный срок не истёк.

Диаграмма параллельного опроса сохранена в diagrams/search_parallel.puml. Внутри фрагмента par одновременно выполняются чтение БД и запросы к трём поставщикам. Ошибка одного поставщика не отменяет результаты остальных.

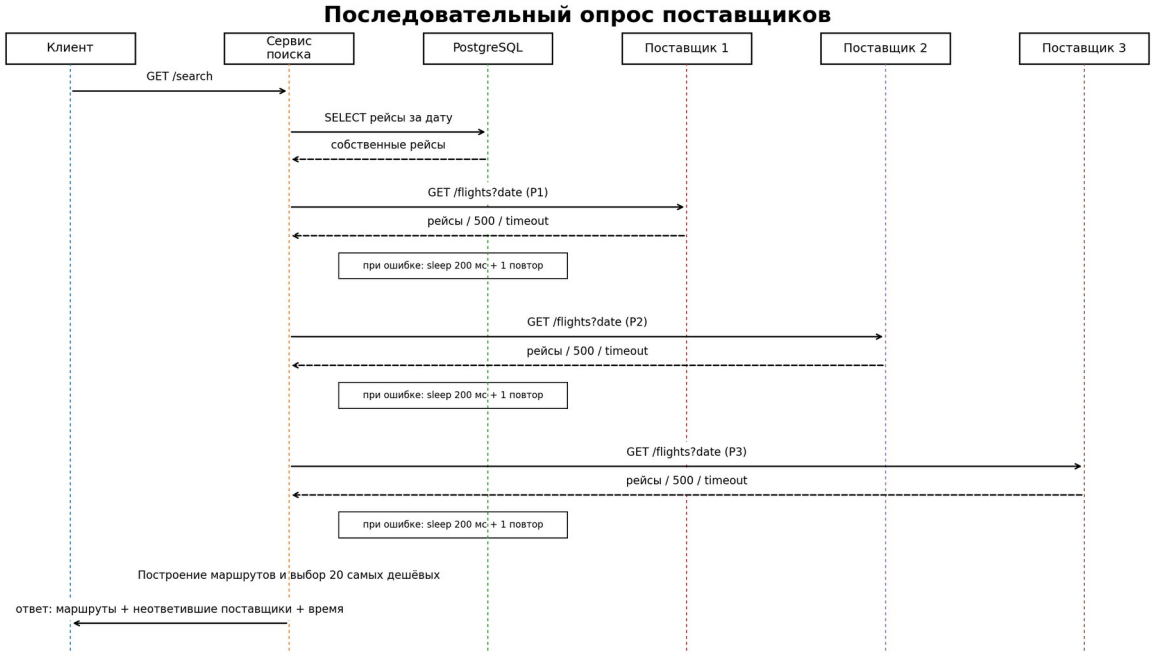


Рисунок 1 — Последовательный опрос поставщиков

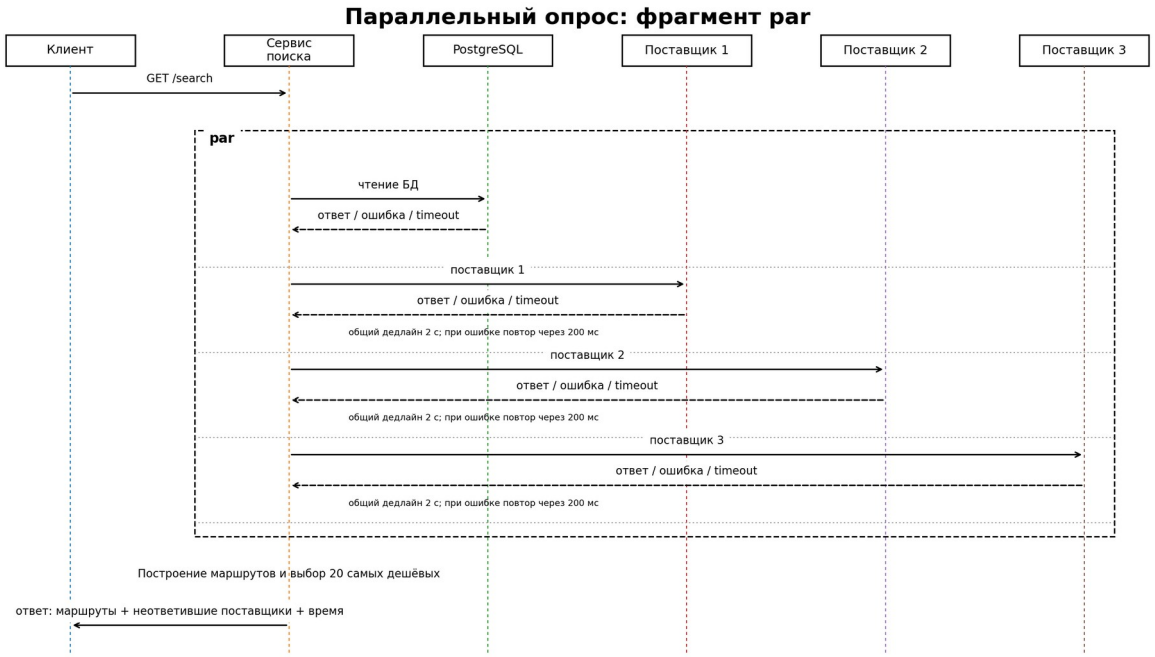


Рисунок 2 — Параллельный опрос поставщиков, фрагмент par

Операция	Ожидаемое среднее время
Поставщик 1, с повторами	325 мс
Поставщик 2, с повторами	535 мс
Поставщик 3, с повторами и тайм-аутами	965 мс
Последовательный опрос: БД + поставщики	около 1 830 мс
Параллельный опрос: максимум из БД и поставщиков	около 990 мс
Построение маршрутов и JSON	около 62 мс
Итого: последовательная версия	около 1 892 мс
Итого: параллельная версия	около 1 052 мс

2.3. Закон Литтла

Для синхронной версии среднее время обработки по расчётной модели принимается W = 1,892 с. При интенсивности λ = 5 запросов/с:

L = λ · W = 5 · 1,892 = 9,46 запроса одновременно.

Полученное среднее число одновременно обрабатываемых запросов (около 9,5) меньше лимита 40 синхронных обработчиков FastAPI. Следовательно, по средней нагрузке лимит потоков не должен быть узким местом; это предположение проверяется замерами.

Соединение с БД требуется только на время чтения расписания. При оценке 5 мс на чтение: LБД = 5 · 0,005 = 0,025. Среднее число одновременно занятых соединений с БД составляет около 0,025.

2.4. Предположение о версиях

Версия	Предположение по /search p95 ≤ 3 с	Предположение по /health p95 ≤ 100 мс	Причина
3.3: def + requests + синхронная БД	Вероятно, проходит; расчётный p95 последовательной модели около 2,91 с, близко к пределу	Вероятно, проходит	При 5 запросах/с средняя занятость около 9,5 потоков при лимите 40
4.1: async def с блокирующими вызовами	Не проходит	Не проходит	Блокирующие requests и time.sleep останавливают цикл событий; запросы и /health ждут
4.2: полноценная асинхронная версия	Вероятно, проходит; расчётный p95 около 2,07 с с учётом построения маршрутов	Вероятно, проходит	Ожидание БД и поставщиков выполняется конкурентно, цикл событий освобождается на await

3. Раздел III. Синхронная версия сервиса

3.1. PostgreSQL и генератор расписания

В db/init.sql создаётся таблица flights с полями перевозчика, номера рейса, аэропортов вылета и прилёта, времени вылета и прилёта и цены. Создан индекс idx_flights_departure_at по departure_at для выборки рейсов за дату. Скрипт seed_db.py загружает 120 дней по 1500 рейсов в сутки (всего 180 000). Генератор использует фиксированную строку начального значения random.Random для каждой даты, поэтому повторный запуск даёт те же рейсы. Продолжительность рейса составляет от 1 до 6 часов.

3.2. Заглушка поставщика

provider_stub.py реализует GET /flights?date=… и GET /health. Число рейсов за дату — 500; набор рейсов детерминирован для поставщика и даты. Параметры задаются переменными окружения: средняя задержка, разброс ±30%, вероятность 500, вероятность задержки 3 секунды и её длительность. Значения в docker-compose.yml соответствуют таблице 1 методички. Параметры Compose: provider1 — 300 мс ±30%, 5% ошибок 500, 0% задержек 3 с, 500 рейсов/сутки; provider2 — 500 мс ±30%, 5%, 0%, 500; provider3 — 800 мс ±30%, 5%, 10%, 500.

3.3. Синхронный сервис поиска

service_sync.py содержит обработчики def для GET /search и GET /health. Запросы к поставщикам выполняются последовательно через requests; повтор выполняется один раз после time.sleep(0.2) и входит в общий дедлайн 2 секунды. Работа с PostgreSQL выполняется

через SQLAlchemy Core и psycopg. Функция build_routes находится отдельно в route_builder.py и не обращается к сети или БД.

3.4. Docker Compose и py-spy

docker-compose.yml включает PostgreSQL, три заглушки поставщиков, одноразовый сервис загрузки расписания и сервис поиска. Для PostgreSQL и веб-сервисов настроены healthcheck. Сервис поиска зависит от готовности БД, заглушек и завершения загрузки расписания. Uvicorn запускается с одним worker; команда использует exec, поэтому uvicorn является PID 1. Для контейнера поиска добавлен SYS_PTRACE.

3.5. Нагрузочный скрипт

load.py реализует замкнутый режим с C клиентами и открытый режим с интенсивностью rate и интервалами random.expovariate(rate). Во время нагрузки /health опрашивается каждые 100 мс. Выводятся число запросов, запросы/с, p50, p95, p99, максимум, процент ошибок и те же показатели для /health. Прогрев и измерение задаются параметрами; значения по умолчанию соответствуют ТЗ: 10 с, 60 с, тайм-аут 30 с. Перцентили считаются методом ближайшего ранга.

3.6. Результаты нагрузочных замеров

Таблица ниже предназначена для результатов 60-секундных замеров после 10-секундного прогрева. В среде подготовки отчёта Docker Engine и PostgreSQL отсутствуют, поэтому реальные значения p50/p95/p99, пропускной способности и ошибок не измерялись. Они не заменяются расчётными значениями. Все 15 запусков выполняются командой bash run_benchmarks.sh и сохраняются в results/*.json.

Версия	Режим	Нагрузка	req/s	p50, мс	p95, мс	p99, мс	Max, мс	Ошибки, %	/health p95
Синхр.	Закрытый	C=1	—	—	—	—	—	—	—
Синхр.	Закрытый	C=10	—	—	—	—	—	—	—
Синхр.	Закрытый	C=40	—	—	—	—	—	—	—
Синхр.	Закрытый	C=80	—	—	—	—	—	—	—
Синхр.	Открытый	5 req/s	—	—	—	—	—	—	—
async def блок.	Закрытый	C=1	—	—	—	—	—	—	—
async def блок.	Закрытый	C=10	—	—	—	—	—	—	—
async def блок.	Закрытый	C=40	—	—	—	—	—	—	—
async def блок.	Закрытый	C=80	—	—	—	—	—	—	—
async def блок.	Открытый	5 req/s	—	—	—	—	—	—	—
Асинхр.	Закрытый	C=1	—	—	—	—	—	—	—
Асинхр.	Закрытый	C=10	—	—	—	—	—	—	—
Асинхр.	Закрытый	C=40	—	—	—	—	—	—	—
Асинхр.	Закрытый	C=80	—	—	—	—	—	—	—
Асинхр.	Открытый	5 req/s	—	—	—	—	—	—	—

Профили py-spy для синхронной версии (с --idle и без него) в данной среде не созданы по той же причине. Команды записи SVG и копирования из контейнера приведены в README.md. По графику без --idle анализируется время выполнения CPU-функций; с --idle дополнительно учитывается ожидание сети и БД.

4. Раздел IV. Переход на асинхронную модель исполнения

4.1. async def без замены блокирующих операций

service_async_blocking.py отличается от синхронной версии объявлением обработчиков через async def. Вызовы requests, time.sleep и синхронный SQLAlchemy оставлены блокирующими, как требует первый эксперимент. Вызовы не содержат await, освобождающего цикл событий, поэтому во время блокирующего ожидания основной поток не обрабатывает другие запросы. Команда py-spy dump для запуска под нагрузкой приведена в README.md. Замеры этой версии по таблице 3 требуют выполнения в Docker-окружении.

4.2. Полноценная асинхронная версия

service_async.py использует httpx.AsyncClient и асинхронный движок SQLAlchemy с драйвером asyncpg. Клиент и движок создаются один раз в lifespan и закрываются при остановке приложения. Чтение БД и запросы к трём поставщикам запускаются совместно через asyncio.gather(..., return_exceptions=True). На каждый запрос поставщику действует asyncio.timeout(2), включая паузу asyncio.sleep(0.2) и единственный повтор. Ошибка одного поставщика не прерывает обработку остальных. Функция построения маршрутов остаётся общей и синхронной, так как это вычисления, а не ожидание ввода-вывода.

Повторные замеры версии 4.2 выполняются той же матрицей load.py и сохраняются в results. Фактическое сравнение трёх версий завершается после запуска bash run_benchmarks.sh.

5. Проверка выполнения заданий ТЗ

Пункт	Требование	Статус
Раздел II, 2.1	Таблица операций и оценка затрат времени	Выполнено аналитически
Раздел II, 2.2	Две UML sequence диаграммы и оценка среднего времени	Выполнено; .puml
Раздел II, 2.3	Оценки по закону Литтла	Выполнено расчётом
Раздел II, 2.4	Предположение по трём версиям	Выполнено; проверка фактом зависит от 3.6
Раздел III, 3.1	PostgreSQL таблица, индекс, повторяемый генератор 120 дней	Реализовано; запуск БД здесь недоступен
Раздел III, 3.2	Заглушки поставщиков, параметры среды, рейсы и health	Реализовано
Раздел III, 3.3	Синхронный сервис def/requests/SQLAlchemy Core	Реализовано

Раздел III, 3.4	Compose, healthcheck, зависимости, один worker, py-spy	Конфигурация реализована; не запущена без Docker
Раздел III, 3.5	Нагрузочный скрипт closed/open и health polling	Реализовано; локально не запускался против сервиса
Раздел III, 3.6	15 замеров и два SVG-профиля	Не выполнено: требуется Docker/PostgreSQL и длительная нагрузка
Раздел IV, 4.1	async def с блокирующими операциями и py-spy dump	Код и команда готовы; dump/замер не сняты
Раздел IV, 4.2	httpx, asyncpg, lifespan, gather, timeout, sleep	Реализовано; замеры не сняты
Раздел V	Вопросы к защите	В методичке приведён только заголовок раздела, сами вопросы отсутствуют

Вывод

Подготовлены исходный код агрегатора, три варианта исполнения, база PostgreSQL и генератор данных, три заглушки поставщиков, построение маршрутов, Docker Compose, нагрузочный скрипт, UML-диаграммы и инструкции для py-spy. Синтаксис Python проверен, девять автоматических тестов прошли. Фактические нагрузочные значения и профилировочные графики остаются единственной незавершённой частью: в среде подготовки отсутствуют Docker Engine и PostgreSQL. Для завершения этой части необходимо выполнить проект на машине с Docker командой bash run_benchmarks.sh и снять указанные профили.